

StoryForge AI — Project Report

Course: Agentic AI

Members: Hamza Bin Riaz 22i-1165 Farhan Ahmed 22i-1200

Table of Contents

1. Project Overview
 2. System Architecture
 3. Technology Stack
 4. Phase 1 — Story Generation Agent
 5. Phase 2 — Audio Generation Agent
 6. Phase 3 — Video Composition Agent
 7. Phase 4 — API and Studio Dashboard
 8. Phase 5 — Edit Agent
 9. MCP Tool Layer
 10. State Management and Version Control
 11. Testing
 12. Sample Output
 13. Setup and Execution
 14. Conclusion
 15. Individual Contributions
-

1. Project Overview

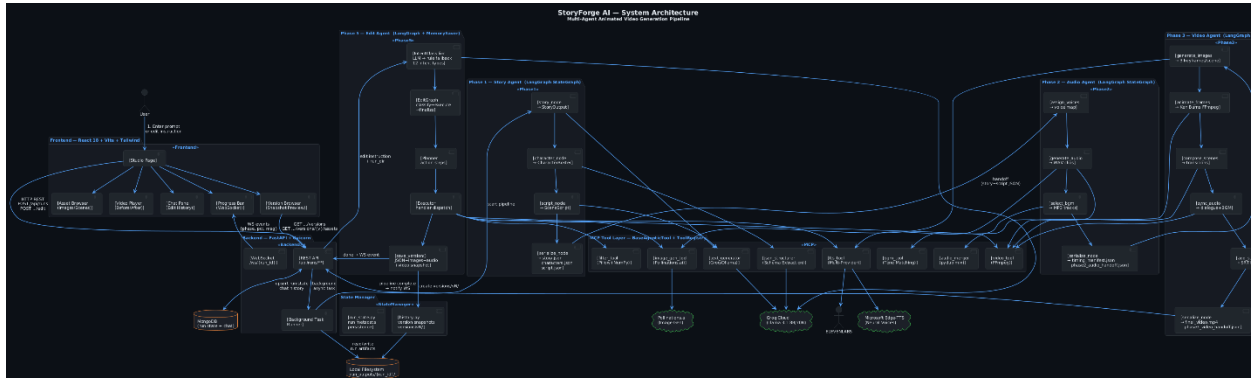
StoryForge AI is an end-to-end multi-agent pipeline that transforms a single natural-language text prompt into a fully composed animated short film. The system requires no manual creative work — it autonomously generates story arcs, characters, dialogue scripts, voice-acted audio, scene imagery, and final video composition.

After the video is generated, users interact with an AI-powered Studio editing interface. They type plain English instructions (“make scene 3 darker”, “switch to anime style”, “speed up the intro”) and the Edit Agent classifies the intent, applies the change, and saves a versioned snapshot — enabling full undo/revert at any point.

Core capabilities: - Prompt → animated video in a fully automated pipeline - 5-phase agent architecture with LangGraph StateGraph per phase - 12 distinct AI edit operations with intent classification - Version history with per-version asset snapshots - Real-time progress streaming via WebSocket - Persistent chat history per run in MongoDB (with local JSON fallback) - 113 passing unit tests across all 5 phases

2. System Architecture

The system is divided into five sequential phases, each implemented as an independent LangGraph StateGraph agent. Phases communicate through serialised JSON handoff files stored in a per-run directory (`run_outputs/{run_id}/`).



All phases share a common MCP (Model Context Protocol) tool layer that abstracts external API calls, file I/O, and media processing behind a uniform `BaseAgenticTool` interface.

3. Technology Stack

Component	Technology	Purpose
Agent Framework	LangGraph 0.x + LangChain	StateGraph per phase, MemorySaver checkpointer
LLM Provider	Groq (Llama 3.1 8B / 70B)	Story generation, intent classification
Image Generation	Pollinations.ai	Free HTTP image API, no key required
Text-to-Speech	Edge-TTS (Microsoft Neural)	High-quality free TTS with emotion support
Premium TTS	ElevenLabs	Optional paid TTS with custom voice IDs
Background Music	Local MP3 assets	Tone-matched BGM per scene
Video Processing	FFmpeg	Ken Burns animation, audio sync, subtitle burn
Image Manipulation	Pillow + NumPy	Filters (sepia, noir,

Component	Technology	Purpose
Backend API	FastAPI + Uvicorn	brightness, warm/cool) REST + WebSocket, async pipeline execution
Database	MongoDB + PyMongo	Run state, message history (JSON fallback)
Frontend	React 18 + Vite + Tailwind CSS	Studio editing UI
Data Validation	Pydantic v2	Typed schemas for all inter- phase handoffs
Testing	pytest 8.x	113 unit tests, no network dependencies

4. Phase 1 — Story Generation Agent

File: agents/story_agent/agent.py

Phase 1 takes a user prompt and produces three structured JSON artifacts: a story arc, a character roster, and a scene-by-scene script.

Graph Structure

story_node → character_node → script_node → serialize_node

Nodes

story_node (story_node.py) - Calls the text_generator MCP tool with the user prompt - Uses json_structurer to parse the LLM output into a StoryOutput Pydantic schema - Validates the narrative arc contains Intro, Climax, and Resolution segments via validate_story_arc() - Estimates total video duration via estimate_duration()

character_node (character_node.py) - Generates a character roster from the story via json_structurer - Runs check_consistency() to fill in missing fields (role, voice_personality, visual_description) with sensible defaults

script_node (script_node.py) - Generates scene-by-scene dialogue using story + characters as context - Runs analyze_emotions() — keyword-based emotion tagging on dialogue lines - Runs build_visual_prompt() — enriches each scene with camera work, character descriptions, and style prefix - Runs assign_transitions() — sets crossfade/flash/fade transitions between scenes - Runs validate_duration() and normalize_scene_ids()

Key Design: Tool Abstraction

All LLM calls go through ToolRegistry.get_tool("text_generator") and ToolRegistry.get_tool("json_structurer"), keeping the agent decoupled from the underlying LLM provider (Groq / Ollama).

5. Phase 2 — Audio Generation Agent

File: agents/audio_agent/agent.py

Phase 2 converts the script into audio — individual WAV clips per dialogue line plus background music per scene.

Graph Structure

assign_voices → generate_audio → select_bgm → serialize_node

Nodes

assign_voices - Maps each character's voice_personality string to an ElevenLabs voice ID or an Edge-TTS voice using keyword matching (e.g. "deep male" → Drew, "warm female" → Rachel)

generate_audio - Iterates over all dialogue lines in the timing manifest - Calls the tts_tool MCP tool with character name, text, emotion, and voice ID - Produces audio/scene_N_line_M_character.wav files - Builds a timing manifest with start_ms / end_ms per clip

select_bgm - Matches scene tone (tense, happy, mysterious, sad, peaceful) to a local BGM asset - Calls bgm_tool which copies the appropriate MP3 to audio/bgm_scene_N.mp3

serialize_node - Writes timing_manifest.json — the contract consumed by Phase 3

TTS Providers (selectable at runtime)

1. Coqui TTS — local, requires Python < 3.12
2. ElevenLabs — cloud, premium quality
3. gTTS — free Google Translate TTS
4. **Edge-TTS — recommended**, free Microsoft Neural voices

6. Phase 3 — Video Composition Agent

File: agents/video_agent/agent.py

Phase 3 generates the final video by combining AI-generated images with audio and motion effects.

Graph Structure

generate_scene_images → animate_frames → compose_scenes
→ sync_audio → add_subtitles → serialize_node

Nodes

generate_scene_images - For each scene, generates 3 keyframes (wide / mid / close) using Pollinations.ai - Each frame is a separate HTTP request to

`https://image.pollinations.ai/prompt/{encoded_prompt}` - Images saved as flat files: `images/scene_N_wide.png`, `images/scene_N_mid.png`, `images/scene_N_close.png`

animate_frames - Applies Ken Burns motion to each still image using FFmpeg zoompan filter - Wide frame: scene-assigned camera motion (`pan_right`, `zoom_in`, etc.) - Mid frame: static (for visual contrast) - Close frame: always `zoom_in` (intimacy effect) - Minimum frame duration: 2 seconds

compose_scenes - Concatenates all animated clips into a single silent video - Applies intra-scene crossfade transitions (0.4 s) between wide/mid/close frames - Applies inter-scene transitions (fade/wipe/dissolve) between scenes

sync_audio / add_subtitles - Merges dialogue audio + BGM onto the silent video timeline using FFmpeg - Burns SRT subtitles using FFmpeg subtitles filter

_compute_scene_durations() — key helper - Derives each scene's video duration from the timing manifest dialogue span + 500 ms trailing pause - Falls back to `duration_estimate_sec` for scenes with no dialogue

7. Phase 4 — API and Studio Dashboard

Backend file: `backend/app.py`

Frontend file: `frontend/src/pages/StudioPage.jsx`

REST API Endpoints

Method	Endpoint	Description
POST	<code>/api/runs</code>	Create run, start pipeline in background
WS	<code>/ws/{run_id}</code>	Real-time progress events
GET	<code>/api/runs</code>	List all runs (MongoDB or disk scan)
GET	<code>/api/runs/{run_id}</code>	Full run state
GET	<code>/api/runs/{run_id}/video</code>	Stream MP4
GET	<code>/api/runs/{run_id}/images</code>	List generated images
GET	<code>/api/runs/{run_id}/messages</code>	Chat history
POST	<code>/api/runs/{run_id}/edit</code>	Submit AI edit instruction
GET	<code>/api/runs/{run_id}/versions</code>	Version list
POST	<code>/api/runs/{run_id}/revert/{v}</code>	Restore a version
GET	<code>/api/runs/{run_id}/versions/{v}/assets</code>	Snapshot assets

MongoDB Integration

- Run state persisted via `$set` upsert on every status change
- Message history appended via `$push` (append-only, never loses history)

- Falls back to local `messages.json` / `story.json` files if MongoDB is unavailable

Studio UI Features

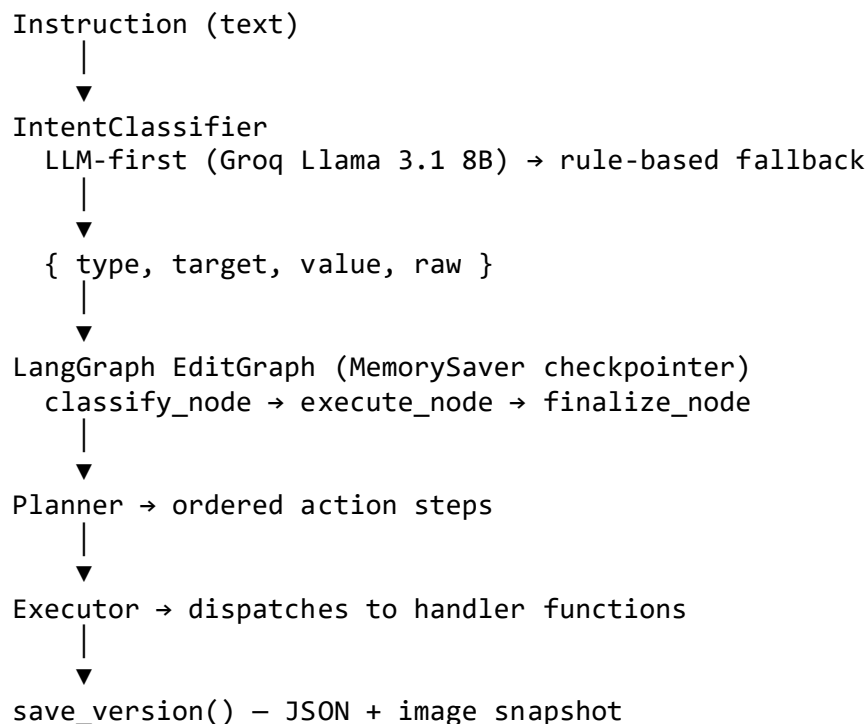
- Real-time pipeline progress bar per phase
- Asset browser (images, characters, scenes) with cache-busting (`?v={refreshKey}`)
- Video player with before/after comparison panel
- Chat panel with persistent AI message history and timestamps
- Version history browser — click any version to preview its assets without reverting
- Amber “Browsing snapshot” banner with one-click restore
- Optimistic message rendering for instant UI feedback

8. Phase 5 — Edit Agent

Files: `agents/edit_agent/agent.py`, `intent_classifier.py`, `planner.py`, `executor.py`

Phase 5 implements a fully autonomous AI editing loop. Users type natural language instructions; the system classifies intent, plans execution steps, applies the edit, and saves a version snapshot.

Architecture



LangGraph Integration

- MemorySaver checkpointer with `thread_id = run_id` — maintains edit history across multiple API calls within the same run
- EditState TypedDict carries: `run_id`, `run_dir`, `instruction`, `intent`, `result`, `edit_history`, `error`

12 Intent Types

Intent	What it does
voice_change	Updates character voice config in characters.json
scene_regen	Deletes existing images, regenerates via Pollinations.ai
style_change	Updates global style tag, regenerates all scene images
bgm_change	Replaces background music track for target scene
script_edit	Rewrites dialogue lines via LLM call
subtitle_remove	Toggles subtitle burn flag in phase3_video_handoff.json
speed_change	Applies FFmpeg setpts filter for speed adjustment
brightness_filter	PIL brightness adjustment on all scene images
character_redesign	Updates visual_description in characters.json
filter_apply	PIL-based sepia / noir / warm / cool colour grading
transition_change	Updates transition type in video handoff JSON
music_add	FFmpeg audio overlay for additional music tracks

Intent Classifier

- **LLM path:** Groq Llama 3.1 8B with a structured JSON schema prompt
- **Rule-based fallback:** Keyword priority chain covering all 12 types
- Priority ordering prevents false positives (e.g. style_change checked before character_redesign to avoid “look like anime” being misclassified)
- Word-boundary regex for ambiguous tokens (e.g. \bart\b avoids matching “start”, “heart”)

9. MCP Tool Layer

Directory: mcp/tools/

All external service calls are wrapped in BaseAgenticTool subclasses and registered with ToolRegistry. This decouples agents from specific APIs and makes tools swappable without changing agent code.

Tool Categories

LLM Tools (llm_tools/) - text_generator.py — Groq/Ollama LLM text generation - json_structurer.py — Schema-guided JSON extraction from LLM output

Audio Tools (audio_tools/) - tts_tool.py — Multi-provider TTS (Edge-TTS / ElevenLabs / gTTS / Coqui) - bgm_tool.py — Tone-to-BGM file mapping - audio_merger.py — Dialogue + BGM mixing via pydub

Vision Tools (vision_tools/) - image_gen_tool.py — Pollinations.ai HTTP image generation - image_edit_tool.py — PIL-based image editing - style_transfer.py — Art style transformation

Video Tools (video_tools/) - ffmpeg_tool.py — FFmpeg wrapper (Ken Burns, concat, speed, subtitles) - compositor_tool.py — Scene assembly and transition rendering - subtitle_tool.py — SRT generation and burn-in

System Tools (system_tools/) - file_tool.py — File read/write abstraction - state_tool.py — State persistence helpers - logger_tool.py — Structured logging

10. State Management and Version Control

Directory: state_manager/

Per-Run State

Each pipeline run has a dedicated directory run_outputs/{run_id}/ containing: - story.json, characters.json, script.json — Phase 1 outputs - timing_manifest.json — Phase 2 timing contract - phase3_video_handoff.json — Phase 3 input spec - images/scene_N_wide.png etc. — Generated keyframes (flat structure) - audio/ — WAV clips and BGM MP3s - video/final_video.mp4 — Composed output - messages.json — Chat history fallback

Version Snapshots (history.py)

Before every edit, save_version() creates a **fully self-contained** point-in-time snapshot — every asset needed to reproduce that exact state is stored inside the version folder:

```
run_outputs/{run_id}/
  versions/
    v1/
      story.json
      characters.json
      script.json
      timing_manifest.json
      images/                                ← all scene keyframes
        scene_1_wide.png
        scene_2_mid.png
```



```

    ...
    audio/                                ← complete audio snapshot
        full_audio.mp3
        mixed/
            scene_1_mixed.mp3
            scene_2_mixed.mp3
        ...
    video/
        final_video.mp4                  ← actual video file copy
        version.json                     ← {version, label, saved_at}
v2/
    ...

```

This enables: - **Complete version isolation** — each version folder is fully independent; browsing v2 loads its own images, audio, and video without touching the live state - **Before/after video comparison** — the video player switches between the snapshot's video/final_video.mp4 and the current live video - **Per-version asset browsing** — Studio UI fetches /versions/{v}/assets which returns images, audio file list, characters, script, and video URL all from the snapshot - **Full revert** — restore_version() copies JSON, images, audio, and video back to run root, fully restoring that state - **Branch edits** — editing while browsing a snapshot sends base_version to the backend, which restores that version first so the edit branches from that historical state rather than the latest

MongoDB Storage (storage.py)

- StateManager.create_run() creates a run document
- StateManager.snapshot() increments version counter
- StateManager.get_state() loads GlobalState Pydantic model from DB

11. Testing

The project has **113 unit tests** across all 5 phases, runnable with zero network calls.

Test Breakdown

Test File	Phase	Tests	What's Covered
tests/unit/test_phase1_agents.py	1	18	validate_story_arc, estimate_duration, check_consistency, analyze_emotions, build_visual_prompt, assign_transitions, validate_duration, normalize_scene_ids

Test File	Phase	Tests	What's Covered
tests/unit/test_schema.py	1	6	Pydantic schema validation for all data models
tests/unit/test_story_agent.py	1	3	LangGraph graph construction, mock full run
tests/unit/test_phase2_audio.py	2	6	assign_voices voice mapping, serialize_node manifest writing
tests/unit/test_phase3_video.py	3	10	_compute_scene_durations, serialize_node, graph construction
tests/unit/test_phase4_api.py	4	19	_load_json helper, message file helpers, version save/list/restore/meta
agents/edit_agent/tests/test_intent_classifier.py	5	51	All 12 intent types, schema validation, scene target extraction, edge cases
Total		113	

Running Tests

```
# From project root
pytest tests/unit/ agents/edit_agent/tests/ -v
```

All 113 tests pass in under 2 seconds with zero network calls.

12. Sample Output

The following was generated from the prompt: *“Deux frères et sœurs séparés par la guerre se retrouvent”*

Story: “Réunis” — Two siblings separated by war reunite in a poignant setting.

Themes: L’amour fraternel, La résilience

Characters: Léon (Protagonist), Aurélie (Protagonist), Le Narrateur, Le Soldat

Scenes: 5 scenes across war-torn locations

Pipeline output: 15 scene images (5 scenes × 3 frames), 5 audio clips, final video with subtitles

Sample images are available in docs/sample_output/.

13. Setup and Execution

Requirements

- Python 3.10–3.12 (3.13 supported with audioop-lts shim)
- Node.js 18+
- FFmpeg on PATH
- MongoDB (optional)

Steps

1. Install Python dependencies

```
pip install -r requirements.txt
```

2. Configure environment

```
cp .env.example .env
```

Edit .env and add your GROQ_API_KEY

3. Install frontend dependencies

```
cd frontend && npm install && cd ..
```

4. Start backend

```
uvicorn backend.app:app --host 0.0.0.0 --port 8000
```

5. Start frontend (new terminal)

```
cd frontend && npm run dev
```

6. Open <http://localhost:5173>

14. Conclusion

StoryForge AI demonstrates a complete agentic AI system built on modern LLM orchestration primitives. Key engineering achievements:

- **Full pipeline automation** from text prompt to MP4 with no manual steps
- **Multi-agent coordination** using LangGraph StateGraph with typed state, distinct from monolithic LLM chains
- **MCP tool abstraction** allowing each agent to swap providers (TTS, image gen, LLM) without code changes
- **Real-time UX** with WebSocket progress streaming and optimistic UI updates
- **Robust version control** — every edit is reversible, assets are snapshotted per version, before/after video comparison is supported
- **Production-quality testing** — 113 unit tests, all offline, covering every phase

- **Graceful degradation** — MongoDB unavailable? Falls back to JSON files. LLM classifier fails? Rule-based fallback activates.

The system successfully generates animated short films from a single prompt and supports iterative AI-powered editing through a natural language chat interface.

15. Individual Contributions

Phase	Component	Contributor
Phase 1	Story Generation Agent — story_node, character_node, script_node, LLM prompting, Pydantic schema design	Farhan Ahmed
Phase 2	Audio Generation Agent — voice assignment, TTS integration (Edge-TTS / ElevenLabs / gTTS / Coqui), BGM selection, timing manifest	Farhan Ahmed
Phase 3	Video Composition Agent — image generation (Pollinations.ai), Ken Burns animation (FFmpeg), scene composition, audio sync, subtitle burn	Hamza
Phase 4	FastAPI backend, REST + WebSocket API, MongoDB integration, React Studio UI, asset browser, video player, real-time progress	Hamza
Phase 5	Edit Agent — LangGraph MemorySaver, IntentClassifier (LLM + rule fallback), 12 edit handlers, version history, branch edits	Hamza

Farhan Ahmed

- Designed and implemented the **Story Generation pipeline** (Phase 1), including the 3-node LangGraph graph, JSON schema for story.json / characters.json / script.json, and all LLM prompt engineering for narrative structure, character creation, and scene scripting.

- Designed and implemented the **Audio Generation pipeline** (Phase 2), including the multi-provider TTS tool (Edge-TTS as primary with ElevenLabs / gTTS / Coqui fallbacks), emotion-to-prosody mapping, voice personality keyword matching, BGM tone selection, pydub-based audio mixing, and the `timing_manifest.json` schema consumed by Phase 3.

Hamza

- Designed and implemented the **Video Composition pipeline** (Phase 3), including AI image generation via Pollinations.ai (3 keyframes per scene), Ken Burns motion effects via FFmpeg `zoompan`, scene transitions, dialogue + BGM audio sync, and SRT subtitle burn.
- Designed and implemented the **FastAPI backend and React Studio UI** (Phase 4), including all REST endpoints, WebSocket real-time progress streaming, MongoDB persistence with JSON fallback, the full Studio dashboard (asset browser, chat panel, video player, before/after comparison), and version history browser.
- Designed and implemented the **AI Edit Agent** (Phase 5), including the LangGraph StateGraph with MemorySaver checkpointing, LLM-first intent classifier with 12 intent types and rule-based fallback, planner/executor architecture, all 12 edit handler functions, complete version snapshot system (JSON + images + audio + video per version), and branch-edit support for editing from historical snapshots.